Prepare Text Data for Analysis

This example shows how to create a function which cleans and preprocesses text data for analysis.

Copy Command

Text data can be large and can contain lots of noise which negatively affects statistical analysis. For example, text data can contain the following:

- Variations in case, for example "new" and "New"
- Variations in word forms, for example "walk" and "walking"
- Words which add noise, for example stop words such as "the" and "of"
- Punctuation and special characters
- HTML and XML tags

These word clouds illustrate word frequency analysis applied to some raw text data from factory reports, and a preprocessed version of the same text data.



Load and Extract Text Data

Load the example data. The file `factoryReports.csv` contains factory reports, including a text description and categorical labels for each event.

```
filename = "factoryReports.csv";
data = readtable(filename,'TextType','string');
```

Extract the text data from the field `Description`, and the label data from the field `Category`.

```
textData = data.Description;
labels = data.Category;
textData(1:10)
```

```
ans = 10×1 string
    "Items are occasionally getting stuck in the scanner spools."
```

```
"Loud rattling and banging sounds are coming from assembler pistons."
"There are cuts to the power when starting the plant."
"Fried capacitors in the assembler."
"Mixer tripped the fuses."
"Burst pipe in the constructing agent is spraying coolant."
"A fuse is blown in the mixer."
"Things continue to tumble off of the belt."
"Falling items from the conveyor belt."
"The scanner reel is split, it will soon begin to curve."
```

Create Tokenized Documents

Create an array of tokenized documents.

```
cleanedDocuments = tokenizedDocument(textData);
cleanedDocuments(1:10)
```

```
ans =
  10×1 tokenizedDocument:

    10 tokens: Items are occasionally getting stuck in the scanner spools .
    11 tokens: Loud rattling and banging sounds are coming from assembler pistons .
    11 tokens: There are cuts to the power when starting the plant .
     6 tokens: Fried capacitors in the assembler .
     5 tokens: Mixer tripped the fuses .
    10 tokens: Burst pipe in the constructing agent is spraying coolant .
     8 tokens: A fuse is blown in the mixer .
     9 tokens: Things continue to tumble off of the belt .
     7 tokens: Falling items from the conveyor belt .
    13 tokens: The scanner reel is split , it will soon begin to curve .
```

To improve lemmatization, add part of speech details to the documents using `addPartOfSpeechDetails`. Use the `addPartOfSpeech` function before removing stop words and lemmatizing.

```
cleanedDocuments = addPartOfSpeechDetails(cleanedDocuments);
```

Words like "a", "and", "to", and the "the" (known as stop words) can add noise to data. Remove a list of stop words using the `removeStopWords` function. Use the `removeStopWords` function before using the `normalizeWords` function.

```
cleanedDocuments = removeStopWords(cleanedDocuments);
cleanedDocuments(1:10)
```

```
ans =
  10×1 tokenizedDocument:

    7 tokens: Items occasionally getting stuck scanner spools .
    8 tokens: Loud rattling banging sounds coming assembler pistons .
    5 tokens: cuts power starting plant .
    4 tokens: Fried capacitors assembler .
    4 tokens: Mixer tripped fuses .
    7 tokens: Burst pipe constructing agent spraying coolant .
    4 tokens: fuse blown mixer .
    6 tokens: Things continue tumble off belt .
    5 tokens: Falling items conveyor belt .
    8 tokens: scanner reel split , soon begin curve .
```

Lemmatize the words using `normalizeWords`.

```
cleanedDocuments = normalizeWords(cleanedDocuments,'Style','lemma');
cleanedDocuments(1:10)
```

```
ans =
  10×1 tokenizedDocument:

    7 tokens: items occasionally get stuck scanner spool .
    8 tokens: loud rattle bang sound come assembler piston .
    5 tokens: cut power start plant .
    4 tokens: fry capacitor assembler .
    4 tokens: mixer trip fuse .
    7 tokens: burst pipe constructing agent spray coolant .
    4 tokens: fuse blow mixer .
    6 tokens: thing continue tumble off belt .
    5 tokens: fall item conveyor belt .
    8 tokens: scanner reel split , soon begin curve .
```

Erase the punctuation from the documents.

```
cleanedDocuments = erasePunctuation(cleanedDocuments);
cleanedDocuments(1:10)
```

```
ans =
  10×1 tokenizedDocument:

    6 tokens: items occasionally get stuck scanner spool
    7 tokens: loud rattle bang sound come assembler piston
    4 tokens: cut power start plant
    3 tokens: fry capacitor assembler
    3 tokens: mixer trip fuse
    6 tokens: burst pipe constructing agent spray coolant
    3 tokens: fuse blow mixer
    5 tokens: thing continue tumble off belt
    4 tokens: fall item conveyor belt
    6 tokens: scanner reel split soon begin curve
```

Remove words with 2 or fewer characters, and words with 15 or greater characters.

```
cleanedDocuments = removeShortWords(cleanedDocuments,2);
cleanedDocuments = removeLongWords(cleanedDocuments,15);
cleanedDocuments(1:10)
```

```
ans =
  10×1 tokenizedDocument:

    6 tokens: items occasionally get stuck scanner spool
    7 tokens: loud rattle bang sound come assembler piston
    4 tokens: cut power start plant
    3 tokens: fry capacitor assembler
    3 tokens: mixer trip fuse
    6 tokens: burst pipe constructing agent spray coolant
    3 tokens: fuse blow mixer
    5 tokens: thing continue tumble off belt
    4 tokens: fall item conveyor belt
    6 tokens: scanner reel split soon begin curve
```

Create Bag-of-Words Model

Create a bag-of-words model.

```
cleanedBag = bagOfWords(cleanedDocuments)
```

```
cleanedBag =
  bagOfWords with properties:

          Counts: [480×352 double]
      Vocabulary: [1×352 string]
        NumWords: 352
    NumDocuments: 480
```

Remove words that do not appear more than two times in the bag-of-words model.

```
cleanedBag = removeInfrequentWords(cleanedBag,2)
```

```
cleanedBag =
  bagOfWords with properties:

          Counts: [480×163 double]
      Vocabulary: [1×163 string]
        NumWords: 163
    NumDocuments: 480
```

Some preprocessing steps such as `removeInfrequentWords` leaves empty documents in the bag-of-words model. To ensure that no empty documents remain in the bag-of-words model after preprocessing, use `removeEmptyDocuments` as the last step.

Remove empty documents from the bag-of-words model and the corresponding labels from `labels`.

```
[cleanedBag,idx] = removeEmptyDocuments(cleanedBag);
labels(idx) = [];
cleanedBag
```

```
cleanedBag =
  bagOfWords with properties:

          Counts: [480×163 double]
      Vocabulary: [1×163 string]
        NumWords: 163
    NumDocuments: 480
```

Create a Preprocessing Function

It can be useful to create a function which performs preprocessing so you can prepare different collections of text data in the same way. For example, you can use a function so that you can preprocess new data using the same steps as the training data.

Create a function which tokenizes and preprocesses the text data so it can be used for analysis. The function `preprocessText`, performs the following steps:

1. Tokenize the text using `tokenizedDocument`.
2. Remove a list of stop words (such as "and", "of", and "the") using `removeStopWords`.
3. Lemmatize the words using `normalizeWords`.
4. Erase punctuation using `erasePunctuation`.
5. Remove words with 2 or fewer characters using `removeShortWords`.
6. Remove words with 15 or more characters using `removeLongWords`.

Use the example preprocessing function `preprocessText` to prepare the text data.

```
newText = "The sorting machine is making lots of loud noises.";
newDocuments = preprocessText(newText)
```

```
newDocuments =
  tokenizedDocument:

   6 tokens: sorting machine make lot loud noise
```

Compare with Raw Data

Compare the preprocessed data with the raw data.

```
rawDocuments = tokenizedDocument(textData);
rawBag = bagOfWords(rawDocuments)
```

```
rawBag =
  bagOfWords with properties:

          Counts: [480×555 double]
      Vocabulary: [1×555 string]
        NumWords: 555
    NumDocuments: 480
```

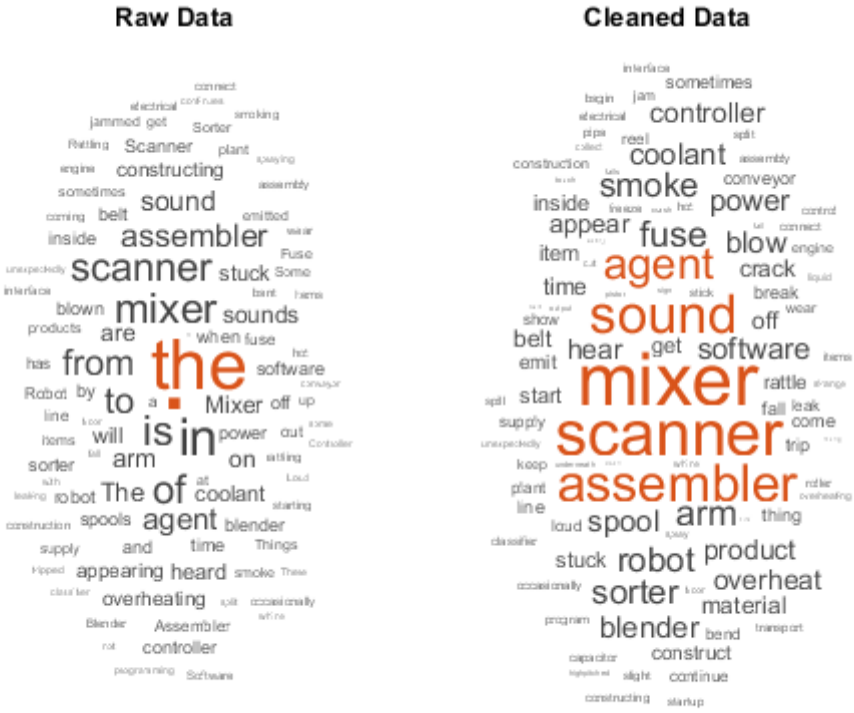
Calculate the reduction in data.

```
numWordsCleaned = cleanedBag.NumWords;
numWordsRaw = rawBag.NumWords;
reduction = 1 - numWordsCleaned/numWordsRaw
```

```
reduction = 0.7063
```

Compare the raw data and the cleaned data by visualizing the two bag-of-words models using word clouds.

```
figure
subplot(1,2,1)
wordcloud(rawBag);
title("Raw Data")
subplot(1,2,2)
wordcloud(cleanedBag);
title("Cleaned Data")
```



Preprocessing Function

The function `preprocessText`, performs the following steps in order:

1. Tokenize the text using `tokenizedDocument`.
2. Remove a list of stop words (such as "and", "of", and "the") using `removeStopWords`.
3. Lemmatize the words using `normalizeWords`.
4. Erase punctuation using `erasePunctuation`.
5. Remove words with 2 or fewer characters using `removeShortWords`.
6. Remove words with 15 or more characters using `removeLongWords`.

```
function documents = preprocessText(textData)

% Tokenize the text.
documents = tokenizedDocument(textData);

% Remove a list of stop words then lemmatize the words. To improve
% lemmatization, first use addPartOfSpeechDetails.
documents = addPartOfSpeechDetails(documents);
documents = removeStopWords(documents);
documents = normalizeWords(documents,'Style','lemma');

% Erase punctuation.
documents = erasePunctuation(documents);

% Remove words with 2 or fewer characters, and words with 15 or more
% characters.
documents = removeShortWords(documents,2);
documents = removeLongWords(documents,15);

end
```

See Also

`tokenizedDocument` | `bagOfWords` | `removeStopWords` | `removeLongWords` | `removeShortWords` | `erasePunctuation` | `removeEmptyDocuments` | `removeInfrequentWords` | `normalizeWords` | `wordcloud` | `addPartOfSpeechDetails`

Related Topics

- Extract Text Data from Files
- Create Simple Text Model for Classification
- Visualize Text Data Using Word Clouds
- Analyze Text Data Containing Emojis
- Analyze Text Data Using Topic Models
- Analyze Text Data Using Multiword Phrases
- Classify Text Data Using Deep Learning
- Train a Sentiment Classifier